

TECHNICAL PROJECT DOCUMENTATION

Delivery Performance, Delay Risk & Logistics Efficiency Analysis

in Global Supply Chain Operations

Dataset: DataCo Smart Supply Chain for Big Data Analysis

Stack: Python · Pandas · Plotly · Streamlit · Scikit-learn · SHAP

Deliverable Type: Data Analytics + Diagnostic Dashboard + Optional ML Layer

Prepared by:

Utpal Chatterjee

1. Executive Overview

This document provides complete technical documentation for the supply chain delivery performance and delay risk analytics project. It covers the problem context, dataset description, architectural design, methodology, KPI definitions, module-level documentation, Streamlit dashboard specification, and delivery checklist.

1.1 Project Objective

Large-scale global logistics operations suffer from reactive rather than preventive delivery management. This project builds a diagnostic intelligence layer that systematically measures on time vs. delayed deliveries, identifies root causes of delay, surfaces high-risk regions and shipping modes, and explains the existing `Late_delivery_risk` indicator in the dataset.

1.2 Business Problem

Despite having detailed order and shipping data, the organization lacks:

- Clear measurement of on time vs. delayed deliveries across regions and shipping modes
- Quantified understanding of why shipments are delayed
- Visibility into which markets, regions, and customer segments carry the highest delay risk
- Structured diagnostics to explain the `Late_delivery_risk` label already present in the data

As a result, logistics teams operate reactively — responding to SLA violations after they occur rather than predicting and preventing them.

1.3 Solution Architecture

The solution is a three-layer architecture:

- Data Pipeline Layer: raw data cleaning, validation, and feature engineering (Delay Gap + Delivery Classification)
- KPI Engine Layer: centralized metric computation functions reused across notebooks and the dashboard
- Dashboard Layer: a multi-page interactive Streamlit application with four analytics modules and live filters
- (Optional) Predictive ML Layer: RandomForest classifier + SHAP explainability for forward-looking risk scoring

2. Dataset Description

Dataset name: DataCo Smart Supply Chain for Big Data Analysis (Kaggle). Approximate row count: 180,000 order-level records. File: DataCoSupplyChainDataset.csv (must be read with encoding='latin-1' — UTF-8 will throw a UnicodeDecodeError).

2.1 Column Reference

Column Name	Type	Description
Type	Categorical	Payment method (Debit, Credit, Transfer, etc.)
Days for shipping (real)	Integer	Actual days taken to ship the order
Days for shipment (scheduled)	Integer	Planned/scheduled shipping duration
Benefit per order	Float	Net profit earned from a single order
Sales per customer	Float	Total sales value generated by the customer
Delivery Status	Categorical	Current delivery status of the order
Late_delivery_risk	Binary (0/1)	Pre-computed label: 1 = late risk, 0 = on time
Category Id / Name	Integer / String	Product category identifier and name
Customer City/Country/State	String	Customer location fields
Customer Segment	Categorical	Consumer, Corporate, or Home Office
Department Id / Name	Integer / String	Responsible department for the product
Latitude / Longitude	Float	Geo-coordinates of the customer location
Market	Categorical	Global market region (e.g., LATAM, Europe, Pacific Asia)
Order City / Country / Region	String	Delivery location fields
Order Item Discount	Float	Absolute discount applied to the order item
Order Item Discount Rate	Float	Discount rate as a proportion of price
Order Item Product Price	Float	Original price before discount

Column Name	Type	Description
Order Item Profit Ratio	Float	Profit ratio on the order item
Order Item Quantity	Integer	Number of units ordered
Sales / Order Item Total	Float	Total and line-item revenue figures
Order Profit Per Order	Float	Net profit on the entire order
Order Status	Categorical	Complete, On Hold, Suspected Fraud, etc.
Product Name / Price	String / Float	Product identifier and selling price
Shipping Mode	Categorical	Standard Class, First Class, Second Class, Same Day
order date (DateOrders)	Datetime	Date the order was placed
shipping date (DateOrders)	Datetime	Date the order was shipped

2.2 Known Dataset Quirks

- Must be loaded with encoding='latin-1' — UTF-8 fails on special characters in product/customer name fields
- Late_delivery_risk is a pre-computed label in the raw data, not something derived during this project — but it should be cross-validated against the computed Delay_Gap to confirm consistency
- Small number of nulls in Customer Lname, Customer Zipcode, and Product Description — these are non-critical fields and can be left as-is or imputed
- The dataset contains two date columns that enable time-series filtering: order date (DateOrders) and shipping date (DateOrders)

3. Project Architecture & Folder Structure

3.1 Folder Structure

```
supply-chain-delay-analysis/  
├── data/  
│   ├── raw/                ← Place DataCoSupplyChainDataset.csv here  
│   └── processed/          ← Auto-generated cleaned_data.csv  
├── notebooks/  
│   ├── 01_data_cleaning. ipynb  
│   ├── 02_eda_overall_performance. ipynb  
│   ├── 03_shipping_mode_analysis. ipynb  
│   ├── 04_regional_market_analysis. ipynb  
│   ├── 05_customer_segment_analysis. ipynb  
│   └── 06_optional_ml_model. ipynb  
├── src/  
│   ├── __init__.py  
│   ├── data_cleaning.py    ← TASK 2: load, validate, clean  
│   ├── feature_engineering.py ← TASK 3: Delay_Gap + classification  
│   ├── kpi_metrics.py      ← TASK 4: all KPI functions  
│   └── utils.py            ← shared helpers  
├── app/  
│   ├── app.py              ← Streamlit entry point  
│   └── pages/  
│       ├── 1_Delivery_Performance.py  
│       ├── 2_Delay_Risk_Analysis.py  
│       ├── 3_Shipping_Mode_Comparison.py  
│       └── 4_Regional_Market_Heatmaps.py  
├── reports/                ← Paper + executive summary outputs  
├── models/                 ← Saved ML model (optional Phase 9)  
├── requirements.txt  
├── .gitignore  
└── README.md
```

3.2 Dependency Graph

The execution dependency chain is strictly linear — each layer depends on the previous one:

```
Raw CSV → data_cleaning.py → cleaned_data.csv → feature_engineering.py →  
DataFrame with Delay_Gap → kpi_metrics.py → KPI scalars + grouped  
DataFrames → Streamlit pages (import from src/)
```

This single-direction flow ensures KPI logic is defined once in `kpi_metrics.py` and reused identically in notebooks and the dashboard — no divergent computations.

4. Analytical Methodology

4.1 Data Cleaning & Validation (src/data_cleaning.py)

The `clean_data(raw_path)` function performs the following steps in order:

Step	Operation	Rationale
1	Load CSV with encoding='latin-1'	Dataset contains non-ASCII characters in name fields
2	Validate Days for shipping (real) ≥ 0 and Days for shipment (scheduled) ≥ 0	Negative shipping days are physically impossible — indicates data entry errors
3	Drop rows missing critical columns	Days for shipping (real/scheduled), Shipping Mode, Order Region, Market, Late_delivery_risk
4	Standardize text columns	<code>strip ()</code> + <code>title ()</code> normalization on Region, Market, Segment, Shipping Mode, Countries
5	Parse order date and shipping date to datetime	Enables date-range filtering in the Streamlit dashboard
6	Drop exact duplicate rows	Prevents double-counting in KPI aggregations
7	Save to data/processed/cleaned_data.csv	Decouples cleaning from analysis — notebooks load processed file, not raw

4.2 Feature Engineering (src/feature_engineering.py)

The `add_delay_features(df)` function computes the core derived features that the entire analysis is built on:

Delay Gap Formula

```
Delay_Gap = Days for shipping (real) - Days for shipment (scheduled)
```

Delivery_Classification: `Delay_Gap > 0 → Delayed` | `Delay_Gap < 0 → Early` | `Delay_Gap = 0 → On-time`

A sanity check cross-tabulation is printed comparing the derived `Delivery_Classification` against the dataset's pre-computed `Late_delivery_risk` label to surface any disagreements between the two.

4.3 KPI Engine (src/kpi_metrics.py)

All KPI functions take a (filtered) dataframe as their sole input and return a scalar or small aggregated dataframe. They have zero Streamlit dependencies so they can be used identically in notebooks and the live dashboard.

Function	Return Type	Description
on_time_delivery_rate(df)	float (%)	Percentage of rows where Delivery_Classification == 'On time'
average_delay(df)	float (days)	Mean of Delay_Gap across all rows
late_delivery_risk_ratio(df)	float (0–1)	Mean of the binary Late_delivery_risk column
shipping_mode_efficiency(df)	DataFrame	Grouped by Shipping Mode: avg_delay, late_rate, volume
regional_delay_index(df, group_col)	DataFrame	Grouped by Order Region or Market: avg_delay, late_rate, volume
customer_segment_impact(df)	DataFrame	Grouped by Customer Segment: late_rate, avg_delay, avg_sales

4.4 Analysis Modules (Notebooks)

Each Jupyter notebook corresponds to one analytical dimension and imports from src/ rather than re-implementing logic inline:

Notebook	Covers	Key Output
01_data_cleaning	Load, validate, clean, save	Cleaned CSV + before/after summary
02_eda_overall_performance	Baseline KPIs, delay distribution	Scorecard + Delivery_Classification bar chart + Delay_Gap histogram
03_shipping_mode_analysis	Mode-wise delay comparison	Late rate by mode bar chart + written interpretation
04_regional_market_analysis	Region, market, country delay diagnostics	Regional delay index table + plotly choropleth/scatter_geo heatmap

Notebook	Covers	Key Output
05_customer_segment_analysis	Segment-level SLA exposure	Segment impact table + bar chart + SLA risk commentary
06_optional_ml_model	Predictive classification + SHAP	RandomForest metrics + SHAP summary plot + saved model

5. Key Performance Indicators (KPIs)

All KPIs are computed at runtime from the live (filtered) dataframe. No values are hard-coded. Each KPI updates immediately when any sidebar filter changes in the Streamlit dashboard.

KPI	Formula / Source	Description
On-Time Delivery Rate (%)	Derived	Percentage of orders where Delivery_Classification == 'On time'. Primary SLA compliance metric.
Average Delivery Delay (Days)	Derived	Mean of Delay_Gap = Days for shipping (real) – Days for shipment (scheduled). Positive = late, negative = early.
Late Delivery Risk Ratio	Raw Col	Proportion of shipments flagged Late_delivery_risk = 1 in the dataset. Range 0–1.
Shipping Mode Efficiency Index	Grouped	Late rate and average delay broken down by Shipping Mode. Identifies modes of driving SLA violations.
Regional Delay Index	Grouped	Average delay and late rate grouped by Order Region and Market. Identifies geographic risk concentration.
Customer Segment SLA Exposure	Grouped	Late rate and average delay by Customer Segment. Flags whether high-value customers are disproportionately impacted.

6. Streamlit Dashboard Specification

6.1 Entry Point: app/app.py

The main app.py file configures the Streamlit app and provides the four sidebar filters that apply to all pages:

Filter	Widget Type	Applies To	Column Used
Shipping Mode	multiselect (all default)	All pages	Shipping Mode
Region / Market	multiselect (all default)	All pages	Order Region or Market
Customer Segment	multiselect (all default)	All pages	Customer Segment
Date Range	date_input (min/max of dataset)	All pages	order date (DateOrders)

All data loading and transformation functions use `@st.cache_data` to avoid recomputation on every widget interaction. The filtered dataframe is derived by applying all four filter conditions simultaneously before passing any KPI function or chart.

Edge case: when the filtered dataframe has zero rows, every page displays a friendly informational message rather than crashing with an empty-dataframe error.

6.2 Page Modules

Page 1 — Delivery Performance Overview (1_Delivery_Performance.py)

Component	Type	Description
Top KPI row	st.metric × 4	On-Time Rate, Avg Delay, Late Risk Ratio, Total Orders
Delivery Classification Chart	Plotly bar / pie	On-time vs Delayed vs Early count and percentage
Delay Gap Histogram	Plotly histogram	Distribution of Delay_Gap values with a vertical line at 0

Component	Type	Description
Delivery Status Breakdown	Plotly bar	Distribution of the raw Delivery Status column values

Page 2 — Delay Risk Analysis (2_Delay_Risk_Analysis.py)

Component	Type	Description
Late Risk Distribution	Plotly pie / donut	Late_delivery_risk = 0 vs 1 split
Delay Gap by Delivery Status	Plotly box plot	Delay_Gap distribution segmented by Delivery Status
High-Risk Combinations Table	st.dataframe	Top 15 Order Region + Market combos by late_rate descending

Page 3 — Shipping Mode Comparison (3_Shipping_Mode_Comparison.py)

Component	Type	Description
Late Rate by Mode	Plotly horizontal bar	late_rate per Shipping Mode, color-coded by risk level
Average Delay by Mode	Plotly bar	avg_delay per Shipping Mode
SLA Compliance % by Mode	Plotly bar	Inverse of late_rate (on-time %) per mode
Mode Efficiency Table	st.dataframe	Full shipping_mode_efficiency() output including volume

Page 4 — Regional & Market Heatmaps (4_Regional_Market_Heatmaps.py)

Component	Type	Description
Geographic Map	Plotly choropleth or scatter_geo	Order Country colored/sized by late_rate — uses Latitude/Longitude or country names
Market Delay Index	Plotly bar	Late rate by Market (LATAM, Europe, Pacific Asia, etc.)
Region Delay Index	Plotly horizontal bar	Late rate by Order Region, sorted descending
Regional Delay Table	st.dataframe (sortable)	Full regional_delay_index() output with volume column

Page 5 (Optional) — Risk Explainability (5_Risk_Explainability.py)

Only built if the optional ML Phase 9 is included. Contains the SHAP feature importance summary plot and an interactive form for predicting Late_delivery_risk for a hypothetical order using the saved RandomForest model.

7. Optional Machine Learning Extension (Phase 9)

The core project brief explicitly positions this analysis as 'the critical first step before deploying predictive or optimization-based supply chain models.' The ML extension is therefore optional but recommended for stronger portfolio positioning.

7.1 Task

Train a binary classifier to predict `Late_delivery_risk` (0 or 1) using the features available at order placement time — i.e., features the logistics team would know before shipping, not outcomes measured after delivery.

7.2 Feature Set

Feature	Type	Encoding
Days for shipment (scheduled)	Integer	Used as-is
Shipping Mode	Categorical	One-hot encoding
Order Region	Categorical	One-hot encoding
Market	Categorical	One-hot encoding
Customer Segment	Categorical	One-hot encoding
Category Name	Categorical	One-hot encoding
Order Item Quantity	Integer	Used as-is

7.3 Model & Evaluation

- Algorithm: `RandomForestClassifier` (`n_estimators=200`, `random_state=42`)
- Split: 80/20 train/test, stratified by target
- Metrics: Precision, Recall, F1-Score, and Confusion Matrix per class
- Output: `classification_report` printed to console + saved model at `models/late_delivery_risk_model.pkl`

7.4 Explainability with SHAP

After training, SHAP (SHapley Additive exPlanations) TreeExplainer is used to generate:

- Global feature importance summary plot: which features contribute most to late delivery risk across all predictions
- Feature direction: whether higher values of a feature increase or decrease risk (e.g., higher scheduled shipping days → higher risk)
- This directly answers the brief's requirement to produce 'diagnostics to explain Late_delivery_risk'

The SHAP approach is a direct reuse of the explainability methodology applied in the HybridNIDS capstone project — the same TreeExplainer pattern applies here.

8. Setup & Installation

8.1 Prerequisites

- Python 3.10 or higher
- pip (Python package manager)
- Git (for version control)
- DataCoSupplyChainDataset.csv downloaded from Kaggle and placed at data/raw/

8.2 Installation Steps

```
# 1. Clone the repository
git clone https://github.com/<your-username>/supply-chain-delay-
analysis.git
cd supply-chain-delay-analysis
```

```
# 2. Create and activate a virtual environment
python -m venv venv
source venv/bin/activate          # Linux/Mac
venv\Scripts\activate            # Windows
```

```
# 3. Install dependencies
pip install -r requirements.txt
```

```
# 4. Place the raw dataset
# Copy DataCoSupplyChainDataset.csv to data/raw/
```

```
# 5. Run the data cleaning pipeline
python src/data_cleaning.py
# Output: data/processed/cleaned_data.csv
```

```
# 6. Launch the Streamlit dashboard
streamlit run app/app.py
```

8.3 Requirements File

```
pandas
```

```
numpy
matplotlib
seaborn
plotly
streamlit
scikit-learn
shap
openpyxl
```

8.4 Deployment (Streamlit Community Cloud)

The dashboard can be deployed for free via Streamlit Community Cloud at streamlit.io/cloud. Point it at your GitHub repository and `app/app.py` as the entry point. Ensure `data/processed/cleaned_data.csv` is committed to the repository or loaded from a public URL — Streamlit Cloud does not have access to your local filesystem.

9. Deliverables Checklist

#	Deliverable	File / Location	Status
1	Cleaned dataset + cleaning script	data/processed/cleaned_data.csv src/data_cleaning.py	
2	Feature engineering script	src/feature_engineering.py	
3	KPI engine module	src/kpi_metrics.py	
4	EDA notebooks (5 + 1 optional)	notebooks/01 through 05 (06 optional)	
5	Streamlit multi-page dashboard	app/app.py + app/pages/	
6	All 4 sidebar filters functional	Applies across all dashboard pages	
7	Geographic heatmap	Page 4: Regional & Market Heatmaps	
8	Research paper (IEEE-style)	reports/research_paper.docx	
9	Executive summary (1–2 pages)	reports/executive_summary.pdf	
10	GitHub README with run instructions	README.md	
11	ML model + SHAP plots (optional)	models/late_delivery_risk_model.pkl	
12	Deployed live dashboard URL	Streamlit Community Cloud	

10. Coding Standards & Best Practices

10.1 Code Quality Rules

- Every function in `src/` must have a docstring stating its input type, return type, and what it computes
- Inline comments are required for non-obvious logic — e.g., the `Delay_Gap` formula, the three-way classification threshold, the `encoding='latin-1'` argument
- No hard-coded numeric values in KPI computations — everything derived from the dataframe at runtime
- Data loading/transform functions in the Streamlit app must use `@st.cache_data` to avoid recomputation on every widget interaction
- Handle the zero-rows-after-filtering edge case in every Streamlit page with a graceful message, not an uncaught exception
- Use plotly charts (not matplotlib) in the Streamlit dashboard for interactivity — rendered with `st.plotly_chart(fig, use_container_width=True)`

10.2 Data Integrity Rules

- Never invent, simulate, or fabricate data — all charts and tables must be computed from the actual loaded dataset
- Log and print a row-count summary when rows are dropped during cleaning — do not silently discard data
- Print the `Late_delivery_risk` vs. `Delivery_Classification` cross-tabulation during feature engineering as a sanity check
- Derive all categorical filter options from `df[column].unique()` — never hardcode category lists

10.3 Version Control

- Commit the cleaned dataset (`data/processed/cleaned_data.csv`) only if it is under 50MB
- Add `data/raw/` to `.gitignore` — never commit the raw 180k-row CSV
- Use descriptive commit messages per phase: e.g., `'feat: add delay gap feature engineering'`, `'feat: shipping mode comparison dashboard page'`

11. Glossary

Term	Definition
Delay Gap	Days for shipping (real) – Days for shipment (scheduled). Positive = delayed, negative = early, zero = on time.
Late_delivery_risk	Pre-computed binary label in the DataCo dataset. 1 = late delivery risk, 0 = no risk.
Delivery_Classification	Derived three-class label: Delayed / On-time / Early, computed from Delay_Gap during feature engineering.
On-Time Delivery Rate	Percentage of all orders where Delivery_Classification == 'On time'.
Late Delivery Risk Ratio	Proportion of orders with Late_delivery_risk == 1. Ranges from 0 (all on time) to 1 (all late).
Shipping Mode Efficiency Index	Average delay and late rate computed per shipping mode — used to compare performance across Standard Class, First Class, Second Class, Same Day.
Regional Delay Index	Average delay and late rate aggregated per Order Region or Market — identifies geographic concentration of delay risk.
SLA	Service Level Agreement. A contract specifying the committed delivery timeline for orders.
SHAP	SHapley Additive exPlanations. A model interpretability method that assigns each feature as a contribution value for a given prediction.
KPI	Key Performance Indicator. A quantified metric used to evaluate operational performance.
EDA	Exploratory Data Analysis. The process of statistically summarizing and visualizing a dataset before formal modeling.

12. Conclusion & Next Steps

This project establishes a complete diagnostic intelligence layer for global supply chain operations. By systematically measuring delivery timelines, classifying orders into On-time / Delayed / Early categories, and aggregating delay risk across shipping modes, regions, markets, and customer segments, it enables logistics teams to move from reactive firefighting to data-driven operational control.

The Streamlit dashboard provides a self-service analytics tool that logistics managers can use daily — filtering by region, shipping mode, customer segment, and date range to monitor SLA compliance and identify the highest-risk areas for immediate intervention.

12.1 Immediate Next Steps After Build

- Run the cleaning pipeline (`python src/data_cleaning.py`) and verify the printed row-count summary
- Open notebooks 02–05 in order and run them top-to-bottom to validate all KPI functions produce non-null, sensible output
- Launch the dashboard (`streamlit run app/app.py`) and verify all four filters live-affect all four pages
- Deploy to Streamlit Community Cloud and share the URL in your GitHub README

12.2 Future Enhancements

- Predictive Layer (Phase 9): add the RandomForest + SHAP explainability module to produce forward-looking risk scores at order placement time
- Real-time integration: replace static CSV loading with a database or API connector to ingest live order data
- Anomaly detection: flag statistical outliers in delay patterns using Isolation Forest or Z-score thresholding
- Optimization simulation: use historical delay patterns to recommend optimal shipping mode selection per route

End of Documentation